

# Real-Time QRS Detection

using Pan–Tompkins

Digital Signal Processing for Electrophysiology

**Abid Mahbub Bari** ID 2305012

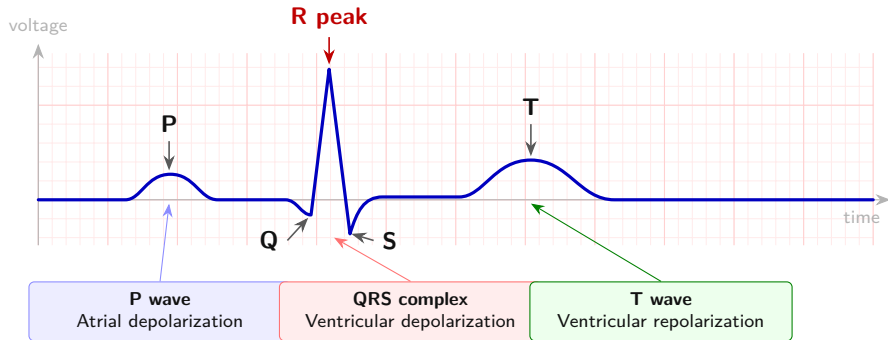
**Shadman Shahriyar Shuvo** ID 2305025

**Let's first look at the signal  
we are working with.**

# The Biological Signal (ECG)

An ECG is a voltage–time trace. One normal heartbeat has three electrically distinct phases:

**Project data:** Our pipeline processes open-source ECG recordings from the MIT–BIH Arrhythmia Database.



**Detection target:** locate each **R peak** accurately despite baseline drift and electrical noise.

**Which ECG morphology  
uniquely identifies ventricular  
depolarization?**

**QRS complex**

# The Engineering Problem: Handling Noisy Data

Raw biological signals are heavily corrupted. The signal is buried in:

- **Baseline Wander ( $< 5$  Hz):** The patient breathing causes the zero-line to slowly drift up and down. (Low-frequency noise)
- **Powerline Interference (50/60 Hz):** Electromagnetic hum from the wall outlet. (High-frequency noise)
- **Muscle Artifacts (EMG):** Random high-frequency spikes from the patient moving.

# The Engineering Problem: Handling Noisy Data

Raw biological signals are heavily corrupted. The signal is buried in:

- **Baseline Wander ( $< 5$  Hz):** The patient breathing causes the zero-line to slowly drift up and down. (Low-frequency noise)
- **Powerline Interference (50/60 Hz):** Electromagnetic hum from the wall outlet. (High-frequency noise)
- **Muscle Artifacts (EMG):** Random high-frequency spikes from the patient moving.

*Conclusion: We cannot use simple voltage thresholds ( $V > 0.5mV$ ). We must process the signal in the **frequency domain** and evaluate its **slope**.*

# Motivation: Why Pan–Tompkins?

---

**Voltage alone is unreliable.** QRS amplitude varies between beats, while noise and tall T waves can produce similar peaks.

**Filtering alone is incomplete.** It suppresses unwanted frequencies, but cannot reliably identify which remaining peak is a QRS complex.

**Pan–Tompkins resolves the ambiguity:**

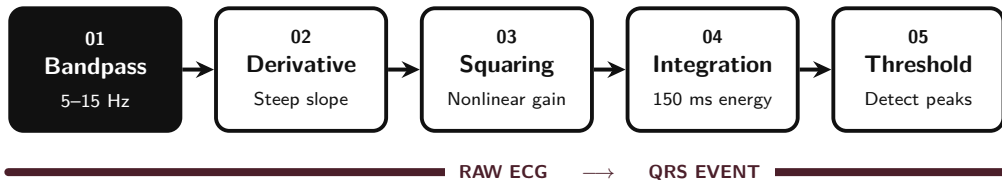
- **Bandpass:** isolates QRS frequencies.
- **Derivative + squaring:** emphasizes rapid slopes, not just height.
- **Integration:** forms one energy pulse per QRS complex.
- **Adaptive threshold:** tracks changing signal and noise levels.

**Result:** a noisy waveform becomes a robust beat event.

**How can we isolate QRS  
energy without amplifying  
physiological and electrical  
noise?**

# The Pan-Tompkins DSP Pipeline

Each stage removes ambiguity until a noisy ECG becomes a reliable beat event.



**Design logic:** filter the frequency band, emphasize rapid change, consolidate its energy, then make the final decision adaptively.

**Given our limited time, we  
will focus on the complete  
pipeline rather than the  
detailed theory behind each  
stage.**

# Key Implementation Decision: The Butterworth Filter

## Stage 1: Bandpass Filter (5-15 Hz)

- **Why 5-15 Hz?** The energy of the QRS complex is concentrated here. This removes both baseline wander and muscle noise.
- **Why Butterworth?** We chose a Butterworth filter because it has a maximally flat frequency response in the passband.
- **Why that matters:** Ripples in the passband (like in a Chebyshev filter) would physically distort the shape of the QRS complex, causing false detections later.

# Mathematical Transformations

## Stage 2: Derivative (Time-Domain)

We use a 5-point difference equation to highlight steep slopes (QRS) and suppress flat waves (P and T).

$$y[n] = \frac{1}{8T} (-x[n-2] - 2x[n-1] + 2x[n+1] + x[n+2])$$

# Mathematical Transformations

## Stage 2: Derivative (Time-Domain)

We use a 5-point difference equation to highlight steep slopes (QRS) and suppress flat waves (P and T).

$$y[n] = \frac{1}{8T} (-x[n-2] - 2x[n-1] + 2x[n+1] + x[n+2])$$

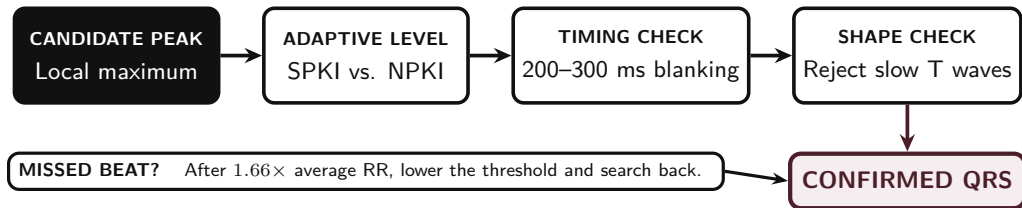
## Stage 3 & 4: Squaring and Moving Window Integration

Squaring makes all points positive and amplifies steep slopes non-linearly. Integration ( $N = 150ms$  window) transforms the sharp QRS spikes into a single, smooth energy pulse for easy thresholding.

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k]^2$$

## Stage 5: Adaptive Decision Logic

A candidate peak is accepted only when its amplitude, timing, and shape are consistent with a QRS complex.



**Adaptive threshold:**  $T = \text{NPKI} + 0.25(\text{SPKI} - \text{NPKI})$ .

**Does the detector maintain  
sensitivity and positive  
predictivity on arrhythmic  
patient data?**

# Benchmarking & Results

---

The original Pan–Tompkins algorithm was evaluated on all 48 recordings in the **MIT–BIH Arrhythmia Database**.

## Evaluation Metrics Explained:

- **Sensitivity (Se):** Did we catch the real beats? ( $TP/(TP + FN)$ )
- **Positive Predictivity (PPV):** Are our detections actually beats, or noise? ( $TP/(TP + FP)$ )

## Original Published Benchmark (1985):

- **Sensitivity: 99.30%**
- **Positive Predictivity: 99.56%** — derived from the published event counts.
- **Failure rate: 0.675%**
- 116,137 total beats, with 507 FP and 277 FN.

**How does a detected QRS complex become a real-time physiological metric?**

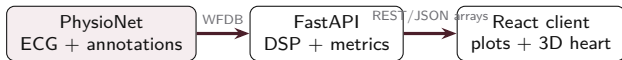
# System Interface & Technology Stack

## Signal-processing backend

- **Python + FastAPI** runs the Pan–Tompkins pipeline and exposes REST endpoints.
- **WFDB** loads 360 Hz MIT–BIH records and annotations from PhysioNet.
- **NumPy/SciPy** performs filtering, convolution, peak detection, and HRV calculations.

## Interactive web frontend

- **React + Vite** provides the application interface and playback controls.
- **Plotly.js** displays the raw ECG and every DSP stage with zooming and a synchronized playhead.
- **React Three Fiber/WebGL** animates the cardiac conduction model at each detected R peak.



**Synchronization:** R-peak timestamps drive the signal playhead, beat animation, heart rate, SDNN, and RMSSD displays.

# Live Demonstration